

Cold Chain Logistics and ShopOps AI

Architecture comparison and implementation reference

Purpose: document the supplied Cold Chain Logistics FDE architecture and use it as a grounded reference for designing ShopOps, an AI e-commerce operations agent.

Prepared as a technical design reference for project planning, implementation sequencing, demos, and interview discussion.

Evidence boundary: Cold Chain statements in this document are limited to the supplied architecture visual and the referenced conversation. Where a capability is not shown, the document says “not specified” rather than inferring an implementation.

Contents

- 1 Executive overview
- 2 Cold Chain Logistics FDE project
- 3 ShopOps AI e commerce operations agent
- 4 Side by side comparison
- 5 Reuse lessons and intentional changes
- 6 Scope from MVP to advanced features
- 7 ShopOps implementation blueprint

1 Executive Overview

The supplied Cold Chain design describes a transition from a traditional logistics setup to an AI powered architecture. The legacy side ingests GPS, truck-sensor, and route-telemetry data into a SQL Server logistics database and exposes static BI reports. The AI side retains the enterprise database, adds an agent that accepts natural-language requests, combines SQL, vector retrieval, and a Weather API, writes to an SQL audit trail, and is deployed through GitHub Actions to AWS EC2.

ShopOps should preserve this architecture pattern while changing the domain from fleet safety and perishability risk to e-commerce support and operations. Its source of truth is the Olist dataset in PostgreSQL, hosted on AWS RDS. Its agent uses LangGraph and LangChain to route natural-language requests to structured-data tools, policy retrieval in Qdrant Cloud, and selected external APIs. Sensitive outcomes such as refunds and compensation require approval before execution and are recorded in an audit log.

Decision	Rationale
Use the Cold Chain project as a structural reference	It demonstrates a clear FDE pattern: preserve the enterprise system of record, add an agentic access layer, connect live context and company knowledge, and make actions traceable.
Keep Cold Chain facts bounded to supplied material	The supplied source names the data sources, SQL Server, agent, Vector RAG, Weather API, audit trail, and AWS CI/CD path. It does not identify frameworks, exact models, vector product, monitoring product, or detailed action semantics.
Make ShopOps deliberately more implementation-specific	The conversation establishes the required ShopOps stack and operational controls, enabling a concrete blueprint without retroactively attributing those choices to Cold Chain.

2 Cold Chain Logistics FDE Project

2.1 Problem and legacy baseline

The Cold Chain visual frames the starting point as a traditional setup where data is stored, visualized, and analyzed manually through static dashboards. GPS data supplies location, route, and speed. Truck sensors supply temperature, fuel, and engine-health signals. Route telemetry supplies distance, stops, and travel time. These feeds enter a Microsoft SQL Server logistics database, which drives static BI reports.

- Manual analysis and queries limit the speed of investigation.
- Static dashboards do not provide real-time reasoning over an operator question.
- The legacy setup has no depicted integration with external live context such as weather.

- Company SOPs, compliance rules, safety guidelines, and best practices are not shown as directly accessible to the legacy dashboard user.
- The diagram identifies a lack of audit trail for user actions and limited scalability and intelligence as limitations.

2.2 Supported target architecture

Component	Supported role in the supplied Cold Chain design
Data sources	GPS data; truck sensors; and route telemetry remain the operational inputs.
Enterprise database	A standard SQL Server database simulates a legacy enterprise system and stores GPS history, sensor data, and route telemetry.
AI agent or orchestrator	Accepts natural language, understands a request, plans and reasons, calls tools including SQL, APIs, and RAG, and returns actionable insights.
Vector RAG	Connects the agent to company SOPs, compliance rules, safety guidelines, and best practices through a vector database.
Weather API	Provides live weather data, including heatwaves and temperature alerts.
SQL audit trail	Records enterprise audit information for the AI enabled workflow. The exact audit schema and event payload are not specified.
Deployment	A code push to GitHub runs through a GitHub Actions CI/CD pipeline and deploys to AWS EC2 for browser access.

2.3 Cold Chain workflow and data flow

A representative request shown in the visual is: “Show me trucks near Los Angeles. Any spoilage risk?” The request enters the AI agent. The agent can query the SQL Server database for vehicle, sensor, and route context; retrieve relevant company knowledge through Vector RAG; and obtain live weather context from the Weather API. It then returns an actionable answer and writes audit information to SQL. The visual establishes this flow, but it does not specify the exact planning algorithm, query generator, retrieval strategy, model, response format, or how the spoilage-risk calculation is scored.

Flow stage	What is supported by source	What is not specified
1 Request	An end user asks a natural-language logistics question.	User interface technology, authentication, and session design.
2 Structured lookup	The agent calls SQL against the SQL Server logistics database.	Database schema, permissions, SQL safety controls, and query examples.
3 Knowledge retrieval	The agent can use RAG over SOPs, compliance rules, safety guidelines, and best practices.	Embedding model, chunking, retrieval quality checks, and vector database vendor.
4 Live context	The agent can call a Weather API for live weather, heatwaves, and temperature alerts.	Weather provider, location resolution method, failure handling, and caching.
5 Output and record	The agent gives actionable insights; an SQL audit trail is present.	Approval gates, action execution, detailed audit fields, and retention policy.

2.4 AI agent and RAG layer

The source calls the central component an “AI Agent (Orchestrator).” Its explicitly shown responsibilities are natural-language understanding, planning and reasoning, tool invocation, and delivering actionable insights. The design therefore supports describing it as a tool-using orchestration layer, rather than merely a chatbot. Vector RAG provides organizational knowledge that complements the live operational records in SQL Server.

The source does not name LangChain, LangGraph, a foundation model, a prompt framework, a guardrail framework, or an evaluation system. Those technologies should not be presented as Cold Chain implementation facts. They can, however, be used by ShopOps as modern implementation choices for the same architecture pattern.

2.5 Cloud, deployment, observability, guardrails, and auditability

The Cold Chain visual explicitly supports GitHub, GitHub Actions CI/CD, AWS EC2 deployment, browser access, and an SQL audit trail. It does not visibly name a cloud database service, container runtime, reverse proxy, authentication service, infrastructure monitoring platform, LLM observability platform, human approval control, or evaluation suite. The audit trail is shown, but the events it records are not defined. Accordingly, this reference treats observability and guardrails as not specified for Cold Chain, except for the existence of the audit-trail component.

3 ShopOps AI E Commerce Operations Agent

3.1 Product purpose and users

ShopOps is a web application for an e-commerce support or operations manager. Instead of requiring the user to navigate reports or write SQL, it accepts questions such as “Where is order 10 234?”, “Why is this order delayed?”, and “Can we compensate this customer?” It separates informational answers from consequential operations. An answer can be returned immediately when it is read-only; a refund, compensation, or other state-changing action must be prepared, reviewed, approved, executed, verified, and logged.

3.2 Proposed architecture

Layer	Proposed ShopOps implementation
Experience and access	Next.js UI hosted on Vercel. AWS Cognito authenticates users and provides role-aware access through tokens verified by the backend.
API and orchestration	FastAPI runs behind Nginx on AWS EC2. LangGraph models the stateful workflow; LangChain supplies LLM, retriever, and tool abstractions.
LLM	Gemini or Amazon Bedrock is used for classification, planning, tool selection, synthesis, and policy-grounded explanations. The selected provider should be configurable.
Structured data	Olist e-commerce data is modeled in PostgreSQL and hosted on AWS RDS. It supports orders, customers, sellers, products, order events, payment and review context where relevant.
Unstructured knowledge	Company shipping, refund, compensation, and support policies are chunked and embedded into Qdrant Cloud. The agent retrieves only relevant policy passages for a decision.
External context	A shipping, geolocation, or weather API is called only when an investigation requires live delivery context. Provider choice and data contract are implementation decisions.
Controls and records	Input, output, and tool/action guardrails constrain risky requests. Human approval gates sensitive actions. PostgreSQL stores workflow and audit events.

Layer	Proposed ShopOps implementation
Quality and operations	Langfuse traces AI workflows. CloudWatch monitors infrastructure and application health. Docker and Docker Compose support repeatable local environments; GitHub Actions runs tests and deployment checks; Playwright covers end-to-end workflows.

3.3 Core user workflows

Workflow	Agent path	Expected outcome
Order status	Intent classification -> get_order tool -> RDS PostgreSQL -> response synthesis.	A concise status, seller, delivery estimate, and current fulfillment state. No RAG or external call unless needed.
Delay investigation	Order and event lookup -> optional shipping or weather context -> retrieve delivery policy -> synthesize evidence.	An explanation grounded in order facts and relevant policy. The response distinguishes current status from policy eligibility.
Compensation proposal	Read order -> retrieve compensation policy -> compute eligibility and amount -> create approval request.	A recommendation with the reason, proposed amount, policy evidence, and approve or reject choice. No money-like action before approval.
Approved action	Approval token and role check -> action tool -> database transaction -> verification -> audit record.	A confirmed result or recoverable failure state, including approver and operation identifiers.
Bulk high impact request	Scope and eligibility analysis -> safety threshold -> review batch -> explicit approval.	The agent does not automatically execute a broad financial action. It prepares a reviewable batch.

3.4 LangGraph workflow design

The ShopOps graph should represent explicit state rather than relying on an unconstrained prompt loop. A practical state object includes request text, authenticated user and role, selected tools, retrieved policy excerpts, structured records, proposed action, approval status, execution result, and trace identifiers. This makes decision points inspectable and testable.

- Classify the request as read-only investigation, policy question, proposed action, or prohibited request.
- Route to least-privileged tools. A status lookup may call read tools; a compensation request may only prepare a proposal until a human approval event is present.
- Retrieve policy before producing any eligibility recommendation. Retrieved text is untrusted reference data, never executable instruction.
- Validate tool inputs and results. For example, order identifiers must resolve to the permitted tenant or operating scope before disclosure.
- Persist a normalized event for every state transition that matters: request, tool call, policy evidence, proposal, approval decision, execution attempt, result, and failure.

3.5 Data, retrieval, and API boundaries

AWS RDS PostgreSQL is the transactional system of record. ShopOps should create a small operational schema around the Olist data rather than allowing an LLM to issue arbitrary SQL. Typed backend tools such as get_order, get_delivery_timeline, list_seller_delay_metrics, create_compensation_proposal, and execute_approved_action provide a stable boundary. Parameterized queries, role checks, row scoping, and allow-listed filters protect the database.

Qdrant Cloud serves a different purpose: semantic retrieval over policy documents and operating procedures. Each source document should retain a document ID, version, section, effective date, and chunk ID. Responses should cite or display the policy evidence used in a recommendation. A policy document can influence a proposal but never directly invoke a tool.

3.6 Guardrails, approval, auditability, and observability

Control	ShopOps behavior
Input guardrail	Reject or safely redirect requests for secrets, bulk deletion, unlawful disclosure, or unrestricted exports. Normalize and validate identifiers before tool use.
Retrieved-content safety	Treat reviews, policy text, and retrieved documents as data. Do not follow instructions embedded in them.
Tool guardrail	Use typed tools with allow-listed operations. Read and write tools have separate permissions. The agent cannot construct arbitrary database mutations.
Human approval	Require an authorized approver for refunds, compensation, or batch operations. Bind approval to an immutable proposal snapshot, amount, scope, and expiry.
Audit trail	Record actor, role, request ID, order or batch scope, model and prompt version where appropriate, policy version, tool calls, approval, execution status, timestamps, and error details.
AI observability	Use Langfuse to trace graph nodes, model calls, tool calls, retrieved context, latency, token use, errors, and evaluation labels.
Infrastructure observability	Use CloudWatch for EC2 and application logs, health endpoints, resource signals, and alertable failures.

4 Side by Side Comparison

Dimension	Cold Chain Logistics	ShopOps
Domain	Cold-chain fleet and logistics operations.	E-commerce customer support and operations.
Business data	GPS history, sensor data, and route telemetry in SQL Server.	Olist-derived order, customer, seller, product, fulfillment, and related operational data in PostgreSQL on AWS RDS.
Natural-language question	Example asks for nearby trucks and spoilage risk.	Examples ask for order status, delay reason, policy eligibility, or a compensation proposal.
Agent role	Understand, plan, reason, call SQL, APIs, and RAG, then give actionable insights.	LangGraph-driven orchestration selects typed tools, gathers evidence, proposes actions, waits for approval, and verifies execution.
RAG knowledge	SOPs, compliance rules, safety guidelines, and best practices in a Vector RAG component.	Shipping, refund, compensation, and support policies stored in Qdrant Cloud.
External API	Weather API for live weather, heatwaves, and temperature alerts.	Shipping, geolocation, or weather context when needed for delivery investigation.
Auditability	SQL audit trail is shown; schema and workflow semantics are not specified.	PostgreSQL audit events are explicitly designed around proposals, approvals, executions, and verification.

Dimension	Cold Chain Logistics	ShopOps
Deployment	GitHub -> GitHub Actions CI/CD -> AWS EC2 -> browser-accessible app.	GitHub Actions validates and deploys Dockerized FastAPI to EC2 behind Nginx; Vercel hosts Next.js.
Controls	Guardrails, approval, auth, and monitoring are not specified in supplied source.	Cognito roles, input/output/tool guardrails, approval gates, Langfuse, CloudWatch, and Playwright are part of the proposal.

4.1 Equivalent component mapping

Cold Chain component	ShopOps equivalent	Why the mapping holds
GPS, sensor, route telemetry	Orders, fulfillment events, sellers, customers, products, and payment or review context	Both are structured operational signals used to investigate a business question.
SQL Server logistics database	PostgreSQL on AWS RDS	Both remain the authoritative relational system of record.
AI Agent Orchestrator	LangGraph orchestrator with LangChain tools	Both interpret natural language, plan, call tools, and return an actionable result.
Vector RAG company knowledge	Qdrant Cloud policy knowledge base	Both ground reasoning in organization-specific documents.
Weather API	Shipping, geolocation, or weather API	Both add live context unavailable in the enterprise database.
SQL audit trail	PostgreSQL audit and approval records	Both preserve accountability; ShopOps specifies the lifecycle more concretely.
GitHub Actions to AWS EC2	GitHub Actions to Dockerized EC2 deployment behind Nginx	The CI/CD and compute path remains familiar while adding operational hardening.

5 Reuse Lessons and Intentional Changes

5.1 What ShopOps should learn and reuse

- Retain a structured operational database as the source of truth; do not replace it with a vector store or a general-purpose chatbot.
- Add natural language as an access and orchestration layer above existing operational data, rather than making it an isolated demo interface.
- Combine three evidence types when the question needs them: structured records, company knowledge, and live external context.
- Treat the agent as an orchestrator that chooses tools and synthesizes evidence, not as a component that invents operational facts.
- Make deployment and auditability part of the architecture, not post-project decoration.

5.2 What intentionally changes

Change	Reason
Logistics risk to commerce operations	ShopOps addresses delayed deliveries, policies, customer compensation, and seller performance rather than fleet temperature and spoilage risk.
SQL Server to PostgreSQL on RDS	The conversation specifies PostgreSQL and a cloud-hosted relational

Change	Reason
	database for ShopOps.
Generic agent depiction to LangGraph workflow	ShopOps needs explicit branches, retries, approval pauses, and testable state transitions.
Insight-first to action-with-approval	E-commerce financial or customer-impacting actions require a visible human decision before execution.
Unspecified controls to explicit control plane	ShopOps adds Cognito roles, guardrails, audit event design, Langfuse, CloudWatch, and Playwright because reliability and safe action are project requirements.

6 Scope from MVP to Advanced Features

Stage	Deliverable	Acceptance evidence
MVP	Local Docker Compose application with FastAPI, PostgreSQL, a small Olist subset, a basic Next.js UI, and a read-only order-status tool.	A user can ask for an order status and receive verified structured data. Pytest covers tool behavior.
Agent and RAG	LangGraph routes status, delay, and policy questions. Qdrant Cloud indexes a small policy corpus.	Eval cases verify expected tool selection, correct policy retrieval, and grounded answers.
Safe action loop	Compensation proposal, manager approval, idempotent execution simulation, and audit records.	Playwright proves the full request -> approve -> execute -> confirm flow; rejected or expired approvals cannot execute.
Cloud delivery	AWS RDS, EC2, Nginx, Vercel, Cognito, GitHub Actions, CloudWatch, and Langfuse.	A deployed environment has health checks, logs, role-aware access, and successful CI on pull requests.
Advanced	Seller risk analytics, batch review, external shipping enrichment, retry and recovery policies, richer eval dashboards.	Scenario tests demonstrate bounded retries, review thresholds, failure visibility, and measured quality.

7 ShopOps Implementation Blueprint

7.1 Suggested repository and service boundaries

Area	Responsibility
frontend	Next.js UI for chat or investigation view, evidence display, approval queue, and audit history. Playwright owns end-to-end test flows.
api	FastAPI routes, Cognito token verification, authorization, typed request and response contracts, health endpoints, and transaction boundaries.
agent	LangGraph states and nodes, LangChain integrations, prompt templates, policy retrieval, graph-level validation, and tool routing.
data	PostgreSQL schema, Olist ingestion, migrations, parameterized repositories, audit records, and approval snapshots.
knowledge	Policy-document ingestion, chunking, metadata, embeddings, Qdrant collection configuration, retrieval tests, and version control.

Area	Responsibility
infra	Dockerfiles, Docker Compose, Nginx configuration, GitHub Actions, EC2 deployment scripts, RDS connectivity, CloudWatch configuration, and environment templates.
evals	Curated scenarios for tool selection, retrieval relevance, policy-grounded answers, refusal behavior, approval enforcement, and recovery.

7.2 Delivery sequence

- Model a minimal PostgreSQL schema and import a small, representative Olist subset. Define typed, read-only tools before introducing an LLM.
- Build and test the order-status and delay-investigation APIs. Confirm that results are deterministic and tenant or role checks are enforced.
- Add LangGraph for classification and routing. Keep the first graph small: request validation, read tool, optional retrieval, response validation.
- Add Qdrant policy retrieval with document metadata and a visible evidence section in the response.
- Implement compensation as a proposal only. Add approval, idempotency keys, execution verification, and audit events before any state-changing tool.
- Instrument the graph with Langfuse and the host with CloudWatch. Add error categorization before deployment.
- Containerize local services, add GitHub Actions for linting, unit and integration tests, evals, and Playwright smoke coverage, then deploy the backend to EC2 and frontend to Vercel.

7.3 Evaluation matrix

Dimension	Example test	Pass condition
Tool selection	“Where is order 10 234?”	Calls only the permitted order-status path and does not retrieve policy or invoke a write tool.
Retrieval grounding	“Can this delayed order receive compensation?”	Retrieves the relevant policy version and explains eligibility using both order facts and policy text.
Approval enforcement	Approve one proposal, then replay an approval token.	First execution is idempotently recorded; replay does not create a duplicate action.
Prompt injection resistance	Retrieved review contains instruction-like malicious text.	Agent treats it as data, does not exfiltrate information, and follows system and tool constraints.
Failure recovery	Shipping API returns a transient error.	Graph reports degraded context or retries within a bounded policy; it never fabricates external data.

7.4 Final recommendation

Build ShopOps as a focused, credible learning project rather than a broad autonomous-commerce claim. The strongest demonstration is a complete, traceable loop for a few high-value workflows: investigate an order, ground a policy decision, prepare a compensation proposal, obtain approval, execute once, verify the result, and show the audit trail. This preserves the Cold Chain FDE architecture pattern while making the e-commerce controls, cloud deployment, and evaluation story specific enough to build and defend.